Apuntes para la defensa oral — Problemas 1 y 2

Ecualización local de histograma

Estos apuntes explican problema1_ecualizacion_local.py en relación con la consigna y el informe del TP. El foco está en qué hace cada parte, por qué se hace y cómo explicarla oralmente.

1. ¿Qué pide el problema?

La consigna tiene tres partes:

1. Implementar una función que reciba una imagen y una ventana de tamaño M × N y aplique ecualización local.

2. Usar esa función para revelar e identificar los detalles ocultos de la imagen.

3. Comparar resultados al cambiar el tamaño de la ventana.

La idea central es recorrer la imagen con una ventana deslizante. En cada posición se calcula el histograma de los píxeles de esa ventana y se transforma el nivel de gris del píxel que está en el centro. Se repite el proceso para todos los píxeles.

2. Idea de ecualización de histograma

En una imagen en escala de grises de 8 bits, cada píxel tiene un nivel entre 0 y 255.
El histograma cuenta cuántos píxeles hay de cada nivel. La función de distribución acumulada (CDF, cumulative distribution function) suma esos conteos desde el nivel 0 hasta un nivel dado.

En este código, para el nivel r del píxel central, el nuevo nivel se calcula como:

nuevo nivel = redondear((cantidad de píxeles de la ventana con nivel <= r
                         / cantidad total de píxeles de la ventana) × 255)

Es decir, cdf[r] / (M*N) estima qué fracción de los píxeles de la ventana tiene intensidad menor o igual que la del píxel central. Multiplicar por 255 lleva esa fracción al rango completo de grises.

Intuición: cada píxel se contrasta contra la distribución de intensidades de su propio entorno, en vez de compararlo contra toda la imagen. Por eso pueden aparecer detalles que globalmente quedaban opacados por el fondo dominante.

Nota conceptual: esta implementación usa directamente la CDF acumulada, sin restar el primer valor no nulo de la CDF. Es una variante concreta de mapeo por CDF; conviene describir la fórmula del código tal cual en la defensa.

3. Imports y preparación del entorno (líneas 12–26)

- os permite construir rutas y comprobar si existen archivos o directorios.

- sys permite agregar rutas de paquetes a sys.path.

- El ciclo de las líneas 16–21 busca site-packages en dos ubicaciones posibles de
.venv. Si encuentra una, la agrega a sys.path para que Python pueda importar las
dependencias de ese entorno.

- cv2 (OpenCV) carga y procesa imágenes; numpy crea arreglos y calcula histogramas
acumulados; matplotlib.pyplot genera las figuras; time mide cuánto tarda cada ecualización.

La búsqueda de .venv es una comodidad de ejecución, no forma parte del algoritmo de
procesamiento.

4. La función principal: ecualizacion_local_histograma (líneas 28–70)

4.1 Parámetros y precondición

def ecualizacion_local_histograma(img, ksize=(31, 31),
border_type=cv2.BORDER_REPLICATE):

- img: imagen de entrada, esperada como arreglo bidimensional uint8 en escala de
grises.

- ksize: tamaño de la ventana (M, N). El valor predeterminado es 31 × 31.

- border_type: política para extender la imagen en los bordes; por defecto replica el
píxel de borde.

La aserción assert len(img.shape) == 2 verifica que la imagen sea monocromática:
tiene filas y columnas, no un tercer eje de canales de color. No verifica
explícitamente el tipo de dato ni que M y N sean positivos.

4.2 Tamaño de ventana y padding (líneas 47–52)

M, N = ksize pad_y = M // 2 pad_x = N // 2 padded = cv2.copyMakeBorder(img, pad_y,
pad_y, pad_x, pad_x, border_type)

M es el tamaño vertical y N el horizontal. Se agrega un margen por arriba y por abajo
de M // 2 píxeles, y por izquierda y derecha de N // 2 píxeles. Ese margen permite
centrar una ventana también sobre los píxeles de los bordes sin salir de los límites
del arreglo.

cv2.BORDER_REPLICATE completa el margen copiando los valores de los píxeles más
cercanos del borde. Así no se introducen ceros artificiales que alterarían el
histograma local en el perímetro.

Las ventanas probadas en el script son impares, por lo que tienen un centro único.
Para tamaños pares, M // 2 no representa simétricamente un centro único y la elección
del píxel central necesita una convención adicional.

4.3 Preparar salida y recorrer la imagen (líneas 54–61)

H, W = img.shape total_pixels = float(M * N) img_out = np.zeros((H, W),

dtype=np.uint8)

- H, W son alto y ancho de la imagen original.

- total_pixels es la cantidad de muestras de cada ventana. Se convierte a float para que la división produzca un resultado real.

- img_out reserva una imagen de salida del mismo tamaño, inicialmente llena de ceros, y con el tipo de píxel requerido para una imagen de 8 bits.

Los dos bucles for recorren cada coordenada (i, j) de la imagen original. En cada iteración:

patch = padded[i:i+M, j:j+N]

Se recorta de la imagen extendida el vecindario de tamaño M × N que corresponde a esa posición. Gracias al padding, el recorte siempre queda completo.

4.4 Histograma, CDF y transformación (líneas 63–68)

hist, _ = np.histogram(patch, bins=256, range=[0, 256]) cdf = hist.cumsum() val_centro = patch[pad_y, pad_x] img_out[i, j] = np.uint8(np.round((cdf[val_centro] / total_pixels) * 255.0))

1. np.histogram cuenta los píxeles del parche en 256 intervalos: uno para cada intensidad posible de 0 a 255. El rango [0, 256] incluye el valor 255 en el último bin.

2. hist.cumsum() produce la CDF: en la posición r contiene el número de píxeles del parche con intensidad menor o igual que r.

3. patch[pad_y, pad_x] obtiene el valor de gris del centro de la ventana.

4. Se consulta la CDF para ese valor central, se divide por M*N, se multiplica por 255 y se redondea.

5. np.uint8 convierte el resultado al formato de intensidad de la imagen de salida.

La ventana no transforma todos los píxeles del parche en esa iteración: determina el nuevo valor del píxel central. Luego el centro se desplaza un píxel y se repite el cálculo. Al terminar, se devuelve img_out.

4.5 Costo computacional

Para cada uno de los H × W píxeles se vuelve a construir un histograma de M × N valores. Por eso el procedimiento es costoso: al crecer la ventana, crece el trabajo de cada posición. El código mide experimentalmente ese tiempo para cada tamaño.

5. Función que organiza el análisis (líneas 73–189)

5.1 Localizar y cargar la imagen (líneas 73–86)

ejecutar_analisis_problema1(custom_image_path=None) determina el directorio d
el
script y busca la imagen:

1. Si se pasó una ruta personalizada y existe, la usa.

2. Si no, intenta Imagen_con_detalles_escondidos.tif junto al script.

3. Como alternativa, busca Imagen_con_objetos_ocultos.tiff.

4. Si se había pasado una ruta personalizada pero no se encontró con las cond
iciones
anteriores, vuelve a asignarla para que la lectura produzca un error claro si
 no
puede cargarse.

cv2.imread(..., cv2.IMREAD_GRAYSCALE) carga directamente la imagen en escala
de
grises. Si OpenCV no logra leerla, img queda como None y se lanza FileNotFoun
dError.

La consigna nombra Imagen_con_objetos_ocultos.tiff; en la carpeta y en el REA
DME se
usa Imagen_con_detalles_escondidos.tif. El script contempla ambos nombres com
o
alternativas.

Después imprime las dimensiones y el mínimo/máximo de intensidad. Esos datos
describen la imagen original; no cambian su contenido.

5.2 Comparación global y procesamiento local (líneas 94–107)

eq_global = cv2.equalizeHist(img) ventanas = [(7, 7), (15, 15), (31, 31), (65
, 65),
(101, 101)]

cv2.equalizeHist calcula una ecualización usando el histograma de toda la ima
gen.
Sirve como referencia para contrastar el método local; no reemplaza la funció
n
implementada.

Luego el programa ejecuta la función local para cinco tamaños. time.time() re
gistra
el instante antes y después de cada ejecución y la diferencia dt da el tiempo
transcurrido. Los resultados se guardan en un diccionario cuya clave es el ta
maño de
ventana, por ejemplo resultados_locales[(31, 31)].

5.3 Guardar resultados (líneas 109–115)

Se crea resultados_problema1/ si no existe. cv2.imwrite guarda la imagen orig
inal, la
ecualización global y las cinco salidas locales. El nombre de cada archivo in
corpora
las dimensiones de la ventana.

5.4 Armar la figura comparativa (líneas 117–139)

Matplotlib crea una figura de seis paneles: imagen original, ecualización glo
bal y
resultados locales para 7×7, 15×15, 31×31 y 65×65. En este panel comparativo
no se
incluye 101×101, aunque ese resultado sí se calcula y se guarda como archivo
individual. cmap="gray" muestra los arreglos con una escala de grises y axis(
"off")
quita los ejes. Al final se guarda comparativa_ventanas.png y se cierra la fi

gura.

5.5 Recortar y mostrar los cinco detalles (líneas 141–167)

eq_optima selecciona el resultado de 31×31, que el análisis del informe considera el
mejor balance visual. Si esa clave no estuviera disponible, usa el primer resultado
del diccionario como alternativa.

El diccionario rois define cinco regiones de interés (Regions of Interest, ROI)
usando cortes de NumPy [fila_inicial:fila_final, columna_inicial:columna_final]. A
cada límite se le suma o resta un margen m = 2 píxeles. Se guardan individualmente y
se presentan juntas en un panel de cinco ampliaciones.

Las ubicaciones y objetos identificados son:

1. Superior izquierda: cuadrado concéntrico más claro.

2. Superior derecha: segmento diagonal a 45°.

3. Centro: letra minúscula a.

4. Inferior izquierda: cuatro líneas horizontales paralelas.

5. Inferior derecha: círculo o disco relleno.

Los índices de esos recortes están escritos para la geometría de esta imagen; no son
un detector automático de objetos y tendrían que recalibrarse si cambia la imagen o
su tamaño.

5.6 Conclusiones que imprime el script (líneas 169–189)

El programa informa los cinco detalles y resume la observación cualitativa sobre las
ventanas:

- 7×7: usa muy pocos píxeles (49), así que el histograma local cambia mucho ante
pequeñas variaciones; puede aumentar el ruido y producir halos.

- 15×15: revela objetos, aunque todavía se observa grano y halos.

- 31×31: el informe lo elige como compromiso entre realzar los objetos y no
amplificar excesivamente el ruido.

- 65×65 y 101×101: incorporan una región más amplia del fondo; la transformación se
adapta menos a cada detalle pequeño y se acerca al comportamiento global.

La ventana es un parámetro de escala espacial: una ventana chica responde a cambios
muy cercanos; una grande resume un entorno más amplio. El tamaño óptimo no es
universal: es una conclusión experimental para esta imagen y estos resultados.

6. Punto de entrada del programa (líneas 191–196)

```
if __name__ == "__main__":
```

Este bloque ejecuta el análisis solamente cuando el archivo se corre directamente, no

cuando se importa como módulo. argparse acepta --image para indicar otra imagen. Si
no se proporciona, el argumento vale None y se activa la búsqueda de nombres
predeterminados.

Ejemplos:

python problema1_ecualizacion_local.py python problema1_ecualizacion_local.py
 --image
ruta/a/otra_imagen.tif

7. Cómo contarlo en la defensa oral

“Implementamos una ecualización local con una ventana deslizante. Para cada píxel,
tomamos el parche centrado en él, calculamos el histograma de 256 niveles y su
acumulada. El nuevo nivel del píxel central es la fracción acumulada correspondiente
a su intensidad, escalada al rango 0–255. Extendemos la imagen con replicación de
bordes para que las ventanas de los píxeles perimetrales sigan teniendo tamaño
completo. Después comparamos cinco tamaños de ventana y usamos la ecualización global
como referencia. En esta imagen, 31×31 dio el mejor equilibrio entre revelar los
cinco objetos y evitar el exceso de ruido de ventanas pequeñas o la pérdida de
adaptación local de las grandes.”

8. Preguntas que podrían hacerte

¿Por qué la ecualización global no alcanza?

Porque usa una sola distribución para toda la imagen. Si una región o nivel tonal
dominante concentra la mayoría de los píxeles, los detalles pequeños y de bajo
contraste pueden ocupar muy poco del rango transformado. La versión local calcula una
distribución distinta para cada vecindario.

¿Qué hace la CDF?

Para cada intensidad r, cuenta cuántos píxeles del parche tienen intensidad menor o
igual que r. Al normalizar por M*N, se obtiene una proporción entre 0 y 1; al
multiplicarla por 255 se obtiene una intensidad de salida.

¿Por qué se usa BORDER_REPLICATE?

En los bordes no existe imagen por fuera del dominio. Replicar los valores de borde
completa el vecindario sin agregar intensidades artificiales, como los ceros de un
padding constante.

¿Qué pasa si hago la ventana más chica o más grande?

La chica es más sensible a cambios locales, pero también al ruido. La grande suaviza
las diferencias locales de la distribución porque incluye más contexto y puede perder
contraste en detalles pequeños. Por eso se evalúa empíricamente el tamaño adecuado.

¿Se ecualiza todo el parche en cada paso?

No. El parche sirve para construir la transformación local, pero el código asigna el
resultado solo al píxel central. El siguiente paso mueve la ventana un píxel.

¿Qué significa que sea una ventana M×N?

M es su cantidad de filas y N su cantidad de columnas. La función admite una ventana
rectangular, aunque los tamaños usados en el experimento son cuadrados.

9. Detalles para no confundir en la exposición

- La ecualización global es una comparación hecha con OpenCV; la función propia
implementa la ecualización local.

- El script procesa una imagen en escala de grises y los resultados son arreglos de 8
bits.

- El cálculo local se repite para cada píxel, por eso el tamaño de ventana impacta en
el tiempo.

- La selección de 31×31 y los cinco recortes de objetos son decisiones experimentales
específicas de esta imagen.

- El panel de comparación incluye hasta 65×65; el resultado 101×101 igualmente se
procesa y se guarda.

- La consigna y el archivo usan nombres distintos para la imagen de entrada; el
script contempla ambos.

Problema 2 — Validación de planillas de calificaciones

Esta parte explica problema2_validacion_planillas.py por bloques: detección de la
tabla, análisis de celdas, validación de campos, generación de reportes y ejecución
sobre varias planillas.

1. ¿Qué pide la consigna?

Para cada planilla de imagen, el programa debe:

1. Determinar si cada campo de cada registro cumple restricciones de forma y reportar
OK o MAL.

2. Crear una imagen con el nombre de los alumnos de registros completamente válidos
cuya condición final sea L o R.

3. Escribir un CSV con el resultado de cada campo.

4. Ejecutar el análisis sobre cuatro planillas.

La consigna aclara que no hay que verificar si el dato es académicamente correcto;
solo hay que verificar su estructura. Por ejemplo, el código no comprueba si una nota
es aprobatoria. Comprueba cuántos componentes tiene el campo y si aparecen es

pacios
según la regla correspondiente.

2. Imports y dependencias (líneas 15–28)

- os construye rutas, obtiene nombres de archivo y crea directorios.

- sys se usa, junto con el ciclo de búsqueda de .venv, para facilitar la importación
de dependencias instaladas en el entorno virtual local.

- cv2 procesa imágenes, detecta componentes y dibuja el reporte visual.

- numpy umbraliza y resume valores de píxeles.

- csv escribe las validaciones en formato CSV.

La preparación de .venv es infraestructura de ejecución. Los pasos de visión por
computadora empiezan en las funciones siguientes.

3. Detectar la cuadrícula: detectar_rejilla_tabla (líneas 30–64)

Esta función devuelve las coordenadas de las líneas horizontales (rows) y verticales
(cols) que delimitan la tabla. Así, el programa puede cortar celdas según las líneas
detectadas en cada imagen en vez de depender de coordenadas fijas para todas las
resoluciones.

3.1 Umbralizar las líneas oscuras (líneas 35–36)

```
bin_grid = (img_gray < 80).astype(np.uint8)
```

Cada píxel menor que 80 se convierte en 1 y los demás en 0. La hipótesis es que las
líneas de la tabla son oscuras sobre un fondo claro. astype(np.uint8) produce una
imagen binaria con valores 0 y 1.

3.2 Separar líneas horizontales y verticales (líneas 38–42)

```
k_h = cv2.getStructuringElement(cv2.MORPH_RECT, (40, 1)) k_v =
cv2.getStructuringElement(cv2.MORPH_RECT, (1, 40)) lines_h =
cv2.morphologyEx(bin_grid, cv2.MORPH_OPEN, k_h) lines_v = cv2.morphologyEx(bin_grid,
cv2.MORPH_OPEN, k_v)
```

Se crean dos elementos estructurantes rectangulares:

- (40, 1): largo horizontal y delgado verticalmente; favorece trazos horizontales
largos.

- (1, 40): delgado horizontalmente y largo vertical; favorece trazos verticales
largos.

La apertura morfológica es una erosión seguida de una dilatación usando el elemento
estructurante. En este caso elimina componentes que no tienen continuidad suficiente
en la dirección buscada y conserva principalmente las líneas de la tabla orientadas
en esa dirección. El texto suele desaparecer porque sus trazos no se extienden lo

suficiente como para contener el elemento estructurante.

3.3 Encontrar columnas mediante una proyección (líneas 44–53)

c_sum = np.sum(lines_v, axis=0) c_thresh = c_sum > (0.3 * c_sum.max())

Sumar por axis=0 acumula los unos de cada columna. Una línea vertical larga g
enera
una suma alta en sus columnas. Se umbraliza el perfil usando el 30 % de su má
ximo.

np.diff compara cada valor booleano con el siguiente. Al rellenar con ceros l
os
extremos y buscar transiciones 0 → 1 y 1 → 0, el código obtiene inicio y fin
de cada
grupo contiguo de columnas activas. El promedio de los extremos da una coorde
nada
representativa para cada línea.

El código espera encontrar ocho líneas verticales: los bordes de siete column
as. Usa
la primera y la última como table_left y table_right, y calcula el ancho tota
l de la
tabla.

3.4 Encontrar filas con umbral de ancho (líneas 56–64)

r_sum = np.sum(lines_h[:, table_left:table_right], axis=1) r_thresh = r_sum >
 (0.7 *
table_w)

Se suman los píxeles de cada fila, restringiéndose al ancho de la tabla. Para
 aceptar
una fila como línea divisoria debe cubrir más del 70 % de ese ancho. Ese crit
erio
ayuda a descartar líneas horizontales cortas, por ejemplo subdivisiones parci
ales del
encabezado.

Como con las columnas, np.diff detecta segmentos contiguos y se toma el punto
 medio
de cada segmento. La función devuelve rows, cols como coordenadas enteras.

Supuesto para defender: los umbrales, los tamaños de los elementos estructura
ntes y
la geometría esperada funcionan con las planillas del TP. No es una detección
universal para cualquier formulario arbitrario.

4. Analizar una celda: analizar_celda (líneas 67–110)

Recibe una celda en escala de grises y devuelve tres resultados: cantidad de
caracteres detectados, cantidad estimada de palabras y recortes binarios de c
ada
componente.

4.1 Quitar bordes de celda y binarizar (líneas 78–87)

Se descartan dos filas y dos columnas por cada lado. El margen evita incluir
líneas
de la cuadrícula como si fueran parte del contenido. Si la celda es demasiado
 pequeña
para quitar ese margen, se devuelve (0, 0, []).

cell_bin = (inner < 120).astype(np.uint8)

Los píxeles oscuros —el texto— quedan como 1 y el fondo claro como 0. El umbr
al 120

separa tinta y papel bajo las condiciones de estas imágenes.

4.2 Componentes conectadas y filtrado (líneas 89–95)

```
num_labels, labels, stats, centroids = cv2.connectedComponentsWithStats(
    cell_bin, connectivity=8
)
```

La función agrupa píxeles de primer plano conectados y entrega, entre otras cosas,
estadísticas por componente: posición (x, y), ancho, alto y área. connectivity=8
considera vecinos laterales, verticales y diagonales conectados.

La etiqueta 0 corresponde al fondo, por eso se usa stats[1:] para mirar componentes
de primer plano. Se conservan solo los componentes de área de al menos 3 píxeles
(s[4] >= 3), para eliminar puntos pequeños de ruido. Si no queda ninguno, se devuelve
una celda vacía.

Este filtro por área permite conservar trazos finos que podrían perderse con un
filtro por altura, pero también implica que un punto de puntuación muy pequeño puede
descartarse.

4.3 Ordenar, contar caracteres y recortar (líneas 97–103)

Los componentes se ordenan por coordenada horizontal x. El número de componentes
válidos se toma como número de caracteres y de cada uno se extrae un recorte binario
con sus límites.

Esto funciona con la tipografía de las planillas siempre que cada carácter aparezca
como una componente separada. Si dos caracteres se tocan, podrían formar una sola
componente; si un carácter se divide en piezas desconectadas, podrían contarse como
varios. El código no reconoce qué letra o dígito es: cuenta formas conectadas.

4.4 Estimar palabras mediante espacios (líneas 104–110)

Para cada par de componentes consecutivas se calcula el espacio horizontal entre sus
cajas delimitadoras:

```
gap = x_inicio_siguiente - (x_inicio_actual + ancho_actual)
```

Se considera que hay un espacio entre palabras si gap >= 7. Por eso:

```
n_words = 1 + cantidad_de_gaps_mayores_o_iguales_a_7
```

Una celda con componentes pero sin espacios grandes cuenta como una palabra. Los
espacios son una estimación geométrica basada en el espaciado de esta tipografía; no
se detectan palabras por significado ni se usa OCR.

5. Clasificar la condición: clasificar_condicion_final (líneas 113–144)

Esta función intenta distinguir L, R y A mirando un recorte binario de un carácter.

Primero verifica que tenga dimensiones mínimas; los recortes demasiado chicos se
clasifican como DESCONOCIDO.

5.1 Contar huecos internos (líneas 123–128)

inv = np.pad(1 - char_crop, 1, constant_values=1) num_bg, _, _, _ =
cv2.connectedComponentsWithStats(inv, connectivity=4) huecos = num_bg - 2

El recorte tiene el carácter como unos y el fondo como ceros. 1 - char_crop invierte
esa representación para que el fondo pase a ser primer plano y se pueda contar como
componentes conectadas.

Se agrega un borde de unos alrededor para que el fondo exterior quede claramente
conectado con el exterior del carácter. Con conectividad 4, solo se consideran
vecinos por arriba, abajo, izquierda y derecha. El conteo incluye la región exterior
y la etiqueta de fondo, de ahí la resta de 2 para estimar huecos cerrados interiores.

5.2 Regla de decisión (líneas 129–143)

- Si no hay huecos, devuelve L.

- Si hay hueco, compara masa de píxeles del carácter en mitad superior e inferior.

- Calcula ratio = top_mass / (bot_mass + 1e-5). El pequeño término evita dividir por
cero.

- Si el cociente es mayor que 0.9, devuelve R; de lo contrario, devuelve A.

La idea es usar primero una característica topológica (huecos) y después una
característica de distribución espacial. Es un clasificador geométrico hecho a medida
de las letras de la planilla, no un reconocedor general de caracteres.

Distinción importante: clasificar la letra y validar que el campo tenga un único
carácter son operaciones separadas. Un campo puede ser estructuralmente OK por
contener un carácter aunque el clasificador devuelva una etiqueta equivocada o
DESCONOCIDO.

6. Procesar una planilla: procesar_planilla (líneas 146–253)

Coordina el procesamiento de un archivo completo y produce el CSV y la imagen de no
aprobados.

6.1 Carga, conversión y límites de la tabla (líneas 150–159)

El nombre base del archivo se utiliza para nombrar las salidas. cv2.imread lee la
planilla en color; si falla, se lanza FileNotFoundError. cv2.cvtColor(...,
COLOR_BGR2GRAY) la convierte a escala de grises para la detección y análisis.

La función detectar_rejilla_tabla devuelve los límites. num_data_rows = len(rows) - 2
descuenta las dos líneas que delimitan la región de datos según la estructura
esperada por el código. En el bucle, cada registro usa rows[r_idx+1] y rows[r

_idx+2]
como borde superior e inferior. La implementación espera 20 filas de datos.

6.2 Recorrer registros y extraer campos (líneas 168–203)

Para cada fila, el identificador es el número de registro empezando en 1. Los cortes
de columnas se basan en las coordenadas detectadas:

| Campo | Columnas de la tabla usadas | Regla aplicada

| Legajo | cols[1] a cols[2] | 8 componentes y 1 palabra

| Nombre y Apellido | cols[2] a cols[3] | al menos 2 palabras y entre 1 y 12 componentes

| Parciales 1, 2 y 3 | pares consecutivos desde cols[3] | 1 o 2 componentes y 1
palabra

| Condición Final | cols[6] a cols[7] | 1 componente y 1 palabra

Cada corte se analiza mediante analizar_celda. Para los parciales, un pequeño bucle
repite la misma regla en las tres columnas. En condición final también se conserva el
recorte para aplicar la clasificación L/R/A, pero solo si la celda tiene exactamente
un componente y una palabra.

En concreto, la regla del nombre exige explícitamente al menos dos palabras y entre 1
y 12 componentes. Es una validación de cantidad de componentes/palabras, no de letras
permitidas.

6.3 Consolidar validez y mostrar por terminal (líneas 205–216)

all_ok combina con and la validez del legajo, del nombre, de los tres parciales y de
la cantidad estructural de caracteres en condición final. all(parciales_ok) solo es
verdadero si los tres parciales están bien.

Se imprime un bloque para cada registro, con OK o MAL para cada campo, siguiendo el
formato pedido por la consigna.

6.4 Acumular datos para CSV y selección de no aprobados (líneas 218–236)

Se agrega un diccionario a registros_csv con el ID y el estado de cada campo. El CSV
guarda los resultados de validación, no transcribe el contenido real escrito en la
planilla.

Para el reporte de no aprobados, se exige all_ok y que la letra detectada sea L o R.
Si se cumplen ambas condiciones, se recorta el campo Nombre y Apellido desde la
imagen a color, con un margen interior de dos píxeles, y se guarda junto con el ID y
la condición.

Así, un registro con condición L o R pero con cualquier otro campo inválido queda
excluido de la imagen final, tal como pide la consigna. Una condición A tambi

én queda
excluida aunque el resto del registro sea válido.

6.5 Escribir CSV y crear la imagen final (líneas 238–253)

El archivo validacion_<nombre_base>.csv se escribe con utf-8 y newline="".
csv.DictWriter fija el orden de columnas, escribe el encabezado y luego todos
 los
registros.

Después se llama a renderizar_imagen_no_aprobados, se guarda la imagen result
ante con
el nombre no_aprobados_<nombre_base>.png y se informa cuántos alumnos fueron
incluidos. La función devuelve las filas del CSV y la lista de alumnos no apr
obados
para que el procesamiento cíclico pueda resumirlos.

7. Renderizar la lista: renderizar_imagen_no_aprobados (líneas 256–324)

Esta función construye una imagen nueva tipo panel, con una tarjeta por alumn
o.

7.1 Caso sin alumnos (líneas 261–273)

Si la lista está vacía, genera un lienzo claro de tamaño fijo, escribe el tít
ulo y un
mensaje de que no se encontraron alumnos L o R, y lo devuelve. Esto asegura q
ue se
cree una imagen de salida incluso cuando no haya coincidencias.

7.2 Tamaño del lienzo y encabezado (líneas 275–282)

Si hay alumnos, calcula la altura total según la cantidad de tarjetas y crea
un
lienzo de tres canales. En OpenCV el orden de color es BGR. Dibuja una franja
 oscura
y escribe el título de la planilla.

7.3 Construcción de cada tarjeta (líneas 283–322)

Para cada alumno:

1. Dibuja el fondo y el borde de la tarjeta.

2. Escribe el número de registro con dos dígitos (#01, #02, etc.).

3. Redimensiona el recorte del nombre para que tenga una altura uniforme, con
servando
su proporción aproximada, y lo pega en el lienzo.

4. Dibuja una insignia de condición: rojo para L y naranja para R, con una et
iqueta
textual.

5. Desplaza la posición vertical para dibujar la tarjeta siguiente.

cv2.resize cambia el tamaño del recorte para ajustarlo a la tarjeta. El panel
 es un
reporte visual; no interviene en la clasificación ni en la validación.

8. Ejecución cíclica: ejecutar_validacion_todas_las_planillas (líneas 327–365
)

La función crea resultados_problema2/. Si recibe una lista personalizada de a
rchivos,
procesa esa lista; si no, construye las rutas de grade_sheet_1.png a
grade_sheet_4.png junto al script.

Recorre las planillas. Si una ruta existe, llama a procesar_planilla, calcula cuántos
registros tienen todos sus campos en OK y agrega un resumen con el nombre del
archivo, cantidad de registros, registros válidos completos y alumnos no aprobados
incluidos. Si falta una imagen, imprime una advertencia y continúa con las restantes.

Al final muestra el resumen general y dónde quedaron los resultados. Esto corresponde
al procesamiento cíclico solicitado por el punto d.

9. Punto de entrada y argumentos (líneas 367–372)

El bloque if __name__ == "__main__": hace que el programa se ejecute como aplicación
al invocarlo directamente. argparse acepta --sheets seguido de cero o más rutas. Si
no se especifican rutas, se procesan las cuatro planillas predeterminadas.

python problema2_validacion_planillas.py python problema2_validacion_planillas.py
--sheets grade_sheet_1.png grade_sheet_2.png

10. Cómo contarlo en la defensa oral

“Primero convertimos cada planilla a gris y detectamos su cuadrícula. Umbralizamos
las líneas oscuras y usamos aperturas morfológicas horizontales y verticales para
aislarlas. Con proyecciones de suma encontramos las coordenadas de filas y columnas,
y con ellas recortamos cada campo de cada registro. Dentro de cada celda quitamos un
margen, binarizamos el texto y contamos componentes conectadas; el espacio entre sus
cajas nos permite estimar palabras. Aplicamos las reglas de cantidad y separación que
pide la consigna, sin intentar reconocer ni validar semánticamente los datos.
Guardamos los estados en un CSV. Para el panel visual seleccionamos solo registros
completamente correctos cuya condición haya sido clasificada como L o R. Finalmente
repetimos el flujo sobre las cuatro planillas.”

11. Preguntas que podrían hacerte

¿Por qué no se usan coordenadas fijas para cada celda?

Porque las planillas pueden tener distintas dimensiones y escala. Se detectan las
líneas en cada imagen y se usan sus coordenadas para ubicar las celdas.

¿Qué hace la apertura morfológica?

Es una erosión seguida de una dilatación con un elemento estructurante. El tamaño y
la orientación del elemento hacen que sobrevivan principalmente líneas
suficientemente largas en esa dirección.

¿El programa sabe si una letra es una A, una R o un nombre real?

No hace OCR ni verifica identidad. Cuenta componentes y espacios. Solo intenta
clasificar la forma del carácter de condición final mediante huecos y distribución de

masa.

¿Por qué se quitan dos píxeles alrededor de cada celda?

Para reducir el riesgo de incluir las líneas divisorias de la tabla como componentes
del contenido.

¿Qué significa conectividad 8?

Dos píxeles se consideran conectados si se tocan por un lado o por una esquina. Esto
ayuda a mantener unidos trazos diagonales.

¿Qué valida el CSV?

Guarda OK o MAL por campo y el ID del registro. No guarda el texto reconocido ni
decide si las notas son académicamente correctas.

12. Detalles para no confundir en la exposición

- Las restricciones implementadas son estructurales: cantidad de componentes y
cantidad estimada de palabras.

- El código no reconoce los caracteres de legajo, nombre ni notas; no comprueba el
alfabeto permitido.

- La regla de nombre se implementa como n_w_nom >= 2 y 1 <= n_c_nom <= 12.

- La condición final se valida por tener un componente y una palabra; la clasificación visual L/R/A es un paso separado.

- Para aparecer en el panel de no aprobados, el registro completo tiene que estar
válido y la letra clasificada debe ser L o R.

- El conteo por componentes depende de que los caracteres no se toquen ni se fragmenten de forma problemática.

- Los umbrales de intensidad, área, gap y los elementos estructurantes están calibrados para estas planillas, no para cualquier documento.